

Animation

Team 6

Contents

1. More about **Skinning**

1-1. GPU **Skinning**

1-2. Bone Weight **Heatmap**

2. **Import** Animation from FBX

3. Animation **State Machine**

4. **Reflection**

GPU Skinning

1. **Skinning**은 bone과 vertex마다 행렬 연산을 수행하는 반복적인 작업이다.

☞ GPU로 가속하기 좋다.

2. **Vertex Shader**에서 할 것인가, **Compute Shader**에서 할 것인가?

- **Vertex Shader**

- Instancing이 가능하다.
- 중복 연산이 발생한다.

- **Compute Shader (Skin Cache)**

- Depth pre-pass, shadow pass, main render pass 등에서 여러 번 재사용 가능하다.
- Ray Tracing 연산을 위해 필수적
- 추가적인 메모리 사용이 필요하다.

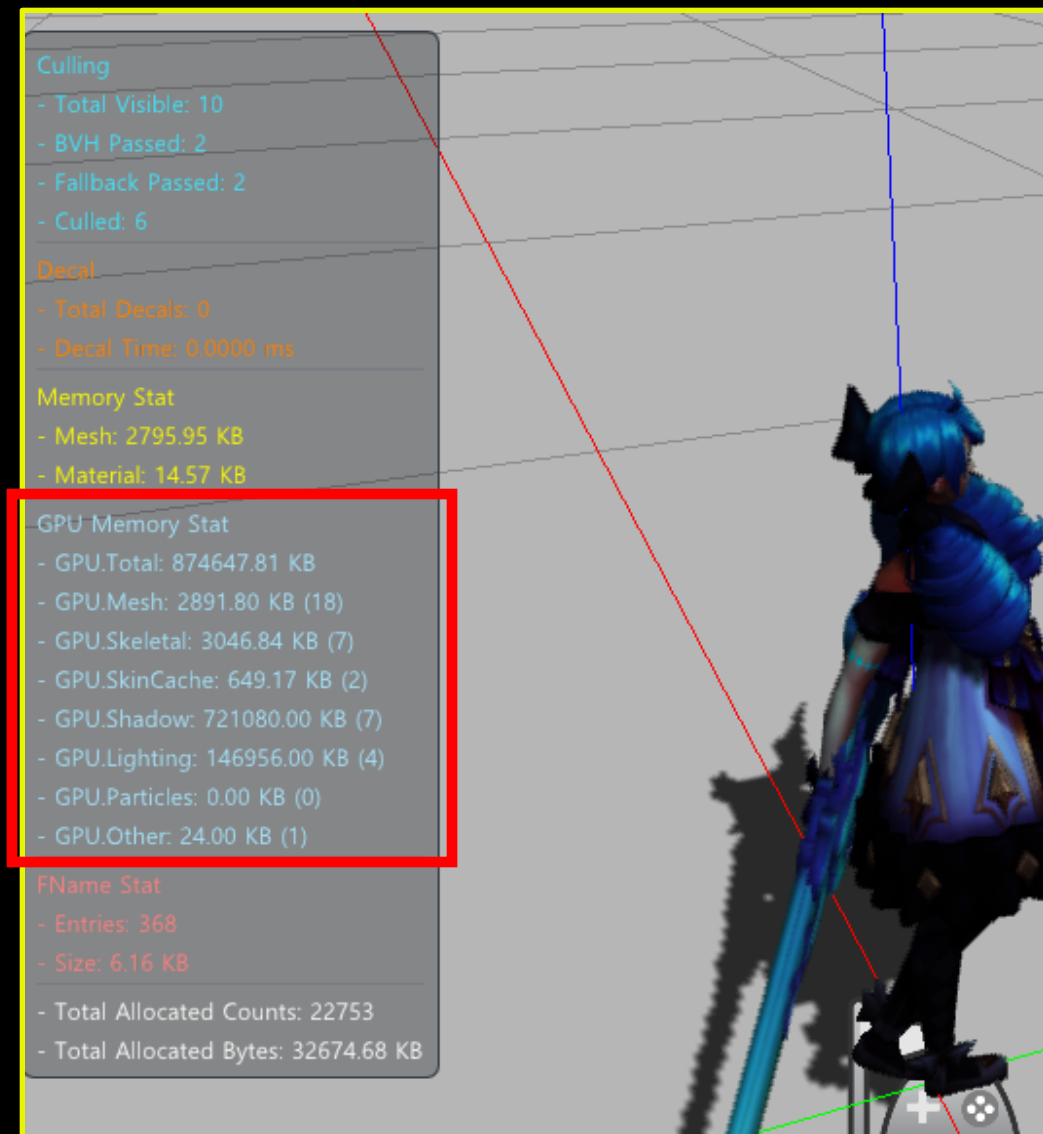
GPU Skinning

3. Per-Component Skin Cache

- 컴포넌트 인스턴스마다 **유저가 skin cache 여부를 선택**할 수 있도록 구현.

4. GPU 메모리 통계

- **Skin cache**를 사용할지 말지 판단할 기준점을 제공한다.



GPU Skinning

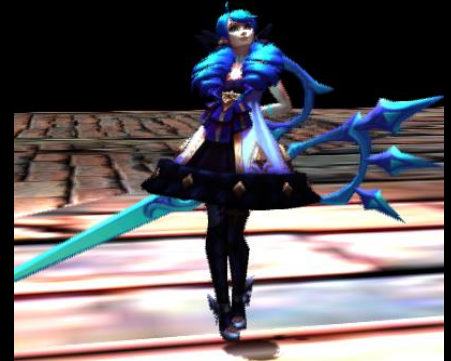
CPU

GPU
(Skin Cache Off)

GPU
(Skin Cache On)

GPU: NVIDIA GeForce RTX 5060
CPU: Intel® Core™ i7-14700
ShadowMap Size: 256x256
ShadowMap Count: 128

Triangle: 13,807
Vertex: 9,636



FrameTime 약 0.7ms 감소

FPS: 38.8 (25.79 ms)

Culling

- Total Visible: 55
- BVH Passed: 2
- Fallback Passed: 3
- Culled: 50

Decal

- Total Decals: 0
- Decal Time: 0.0000 ms

Memory Stat

- Mesh: 48.98 KB
- Material: 15.36 KB

GPU Memory Stat

- GPU.Total: 880355.31 KB
- GPU.Mesh: 143.20 KB (16)
- GPU.Skeletal: 12152.11 KB (24)
- GPU.SkinCache: 0.00 KB (0)
- GPU.Shadow: 721080.00 KB (7)
- GPU.Lighting: 146956.00 KB (4)
- GPU.Particles: 0.00 KB (0)
- GPU.Other: 24.00 KB (1)

FPName Stat

- Entries: 9636
- Size: 201.07 KB

FPS: 91.3 (10.95 ms)

Culling

- Total Visible: 55
- BVH Passed: 2
- Fallback Passed: 3
- Culled: 50

Decal

- Total Decals: 0
- Decal Time: 0.0000 ms

Memory Stat

- Mesh: 48.98 KB
- Material: 15.36 KB

GPU Memory Stat

- GPU.Total: 881376.81 KB
- GPU.Mesh: 143.20 KB (16)
- GPU.Skeletal: 13173.60 KB (27)
- GPU.SkinCache: 0.00 KB (0)
- GPU.Shadow: 721080.00 KB (7)
- GPU.Lighting: 146956.00 KB (4)
- GPU.Particles: 0.00 KB (0)
- GPU.Other: 24.00 KB (1)

FPName Stat

- Entries: 9636
- Size: 201.07 KB

FPS: 97.7 (10.24 ms)

Culling

- Total Visible: 55
- BVH Passed: 2
- Fallback Passed: 3
- Culled: 50

Decal

- Total Decals: 0
- Decal Time: 0.0000 ms

Memory Stat

- Mesh: 48.98 KB
- Material: 15.36 KB

GPU Memory Stat

- GPU.Total: 882026.00 KB
- GPU.Mesh: 143.20 KB (16)
- GPU.Skeletal: 13173.60 KB (27)
- GPU.SkinCache: 649.19 KB (2)
- GPU.Shadow: 721080.00 KB (7)
- GPU.Lighting: 146956.00 KB (4)
- GPU.Particles: 0.00 KB (0)
- GPU.Other: 24.00 KB (1)

FPName Stat

- Entries: 9984
- Size: 208.78 KB

GPU Skinning

CPU

FPS: 2.8 (355.66 ms)

Culling

- Total Visible: 56
- BVH Passed: 2
- Fallback Passed: 4
- Culled: 50

Decal

- Total Decals: 0
- Decal Time: 0.0000 ms

Memory Stat

- Mesh: 48.98 KB
- Material: 25.84 KB

GPU Memory Stat

- GPU.Total: 975107.69 KB
- GPU.Mesh: 143.20 KB (16)
- GPU.Skeletal: 106922.01 KB (5)
- GPU.SkinCache: 0.00 KB (0)
- GPU.Shadow: 721080.00 KB (7)
- GPU.Lighting: 146956.00 KB (4)
- GPU.Particles: 0.00 KB (0)
- GPU.Other: 24.00 KB (1)

FName Stat

- Entries: 2030

GPU
(Skin Cache Off)

FPS: 55.8 (17.93 ms)

Culling

- Total Visible: 56
- BVH Passed: 2
- Fallback Passed: 4
- Culled: 50

Decal

- Total Decals: 0
- Decal Time: 0.0000 ms

Memory Stat

- Mesh: 48.98 KB
- Material: 25.84 KB

GPU Memory Stat

- GPU.Total: 975125.19 KB
- GPU.Mesh: 143.20 KB (16)
- GPU.Skeletal: 106922.01 KB (5)
- GPU.SkinCache: 0.00 KB (0)
- GPU.Shadow: 721080.00 KB (7)
- GPU.Lighting: 146956.00 KB (4)
- GPU.Particles: 0.00 KB (0)
- GPU.Other: 24.00 KB (1)

FName Stat

- Entries: 2030

GPU
(Skin Cache On)

FPS: 63.5 (15.75 ms)

Culling

- Total Visible: 56
- BVH Passed: 2
- Fallback Passed: 4
- Culled: 50

Decal

- Total Decals: 0
- Decal Time: 0.0000 ms

Memory Stat

- Mesh: 48.98 KB
- Material: 25.84 KB

GPU Memory Stat

- GPU.Total: 1007005.31 KB
- GPU.Mesh: 143.20 KB (16)
- GPU.Skeletal: 106922.01 KB (5)
- GPU.SkinCache: 31880.11 KB (2)
- GPU.Shadow: 721080.00 KB (7)
- GPU.Lighting: 146956.00 KB (4)
- GPU.Particles: 0.00 KB (0)
- GPU.Other: 24.00 KB (1)

FName Stat

- Entries: 2030
- Size: 40.11 KB

GPU: NVIDIA GeForce RTX 5060

CPU: Intel® Core™ i7-14700

ShadowMap Size: 256x256

ShadowMap Count: 128

Triangle: 1,035,930

Vertex: 517,972

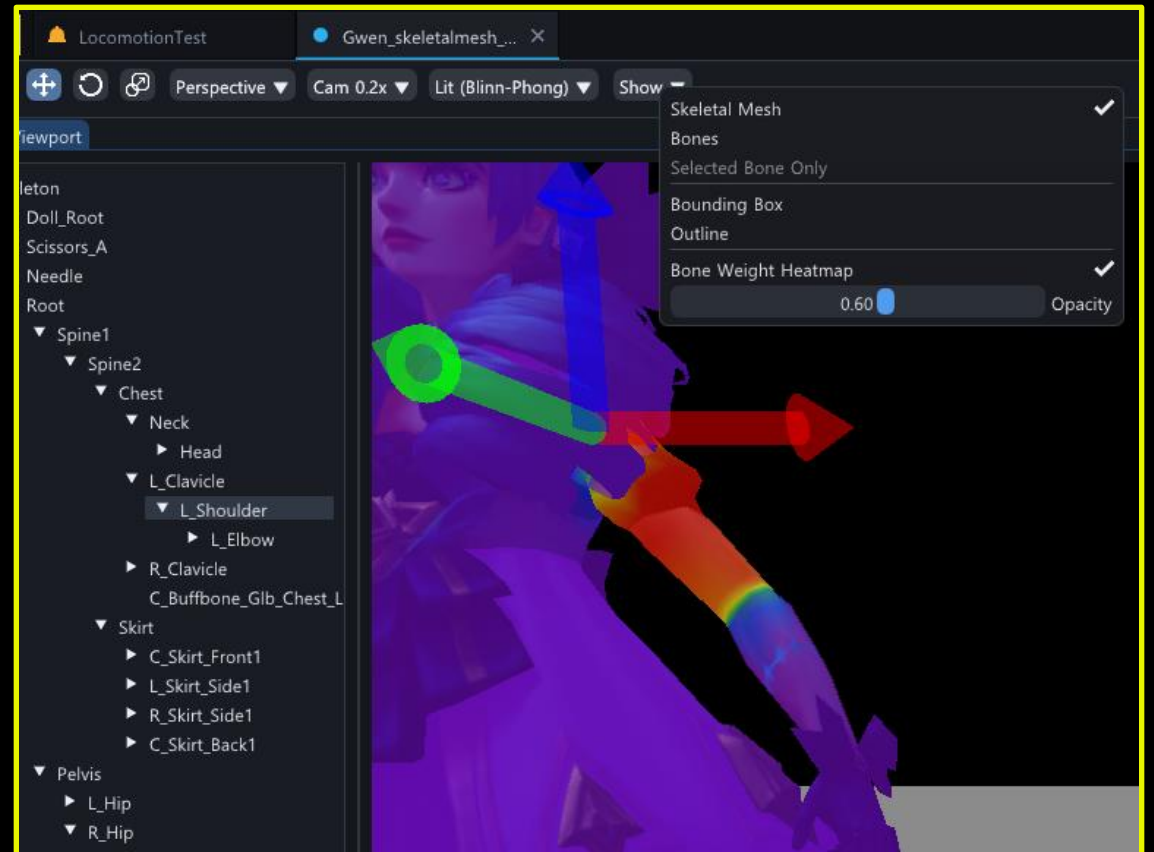


FrameTime 약 2.2ms 감소
High-poly 모델에서
더 효과적인 경향성

Bone Weight Heatmap

1. 임포트한 SkeletalMesh가
제대로 리깅이 되었는지 확인하는 기능.

2. 원본 메시를 확인할 수 있도록
opacity 조절 가능한 overlay로 제공.



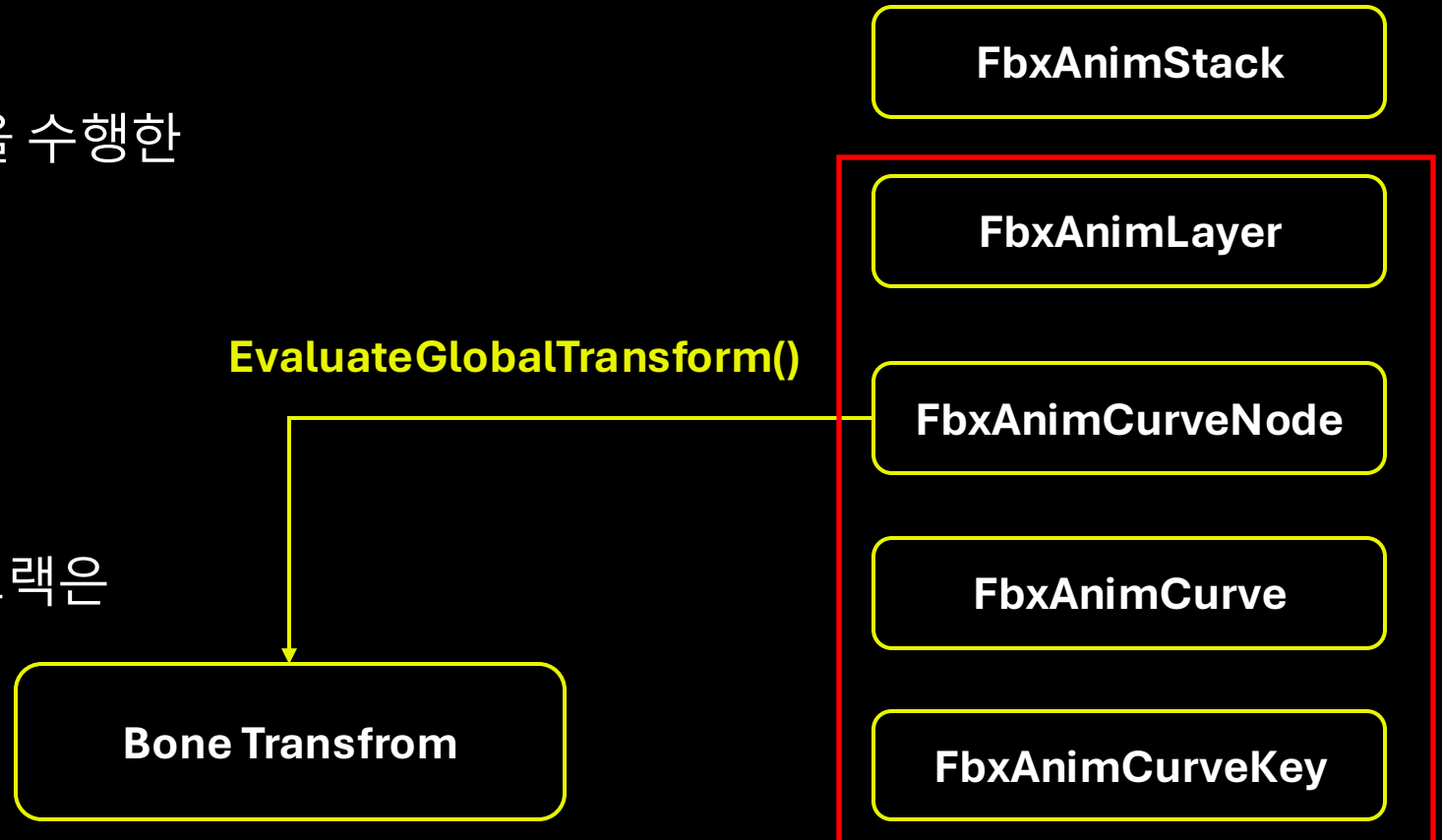
Import Animation from FBX

1. EvaluateGlobalTransform()

- 매 프레임 bone마다 샘플링을 수행한다.

2. 단일 키프레임 압축

- 모든 프레임의 값이 동일한 트랙은 단일 키프레임으로 압축하여 연산량을 절감한다.



Import Animation from FBX

Q. `FbxNode::EvaluateGlobalTransform()`를 왜 사용하는가?

- 매 프레임 bezier curve를 평가하는 것은 비효율적이다.
 - Bezier curve를 평가하기 위해서는 3차 다항식 계산이 필요하다.
 - 반면, 매 프레임 샘플링된 데이터가 있다면 1차식만으로 계산이 가능하다.
- PreRotation, PostRotation, ScalingPivot, RotationPivot 등 수많은 변수들.
- Euler angle로 저장된 회전값을 quaternion 회전값으로 변환한다.

Animation State Machine

Q. 왜 필요한가?

캐릭터 상태를 기준으로 어떤 **AnimSequence**를 재생할지 결정합니다.

1. Gameplay Input -> Anim Parameters

```
const FInputActionState* MoveAction = Snapshot ? Snapshot->FindAction("Move") : nullptr;
const FInputActionState* SprintAction = Snapshot ? Snapshot->FindAction("Sprint") : nullptr;
.
.
.
const bool bLightAttackStarted = IsActionStarted(LightAttackAction);
.
.
.
const bool bMoving = GroundSpeed > MoveStartSpeedThreshold;
```

Animation State Machine

2. Locomotion State 등록

```
LocomotionStateMachine->SetEntryState(FName("Idle"));
LocomotionStateMachine->AddState(FName("Idle"), FName("Idle"), true, IdleAnimationPath);
.
.
LocomotionStateMachine->AddState(NAME_WalkToIdle, NAME_WalkToIdle, false, WalkToIdleAnimationPath);
.
.
LocomotionStateMachine->AddState(NAME_LightAttack1, NAME_LightAttack1, false, LightAttackAnimationPath);
LocomotionStateMachine->AddState(NAME_LightAttack3, NAME_LightAttack3, false, LightAttack3AnimationPath);
```

Animation State Machine

3. Locomotion Transition

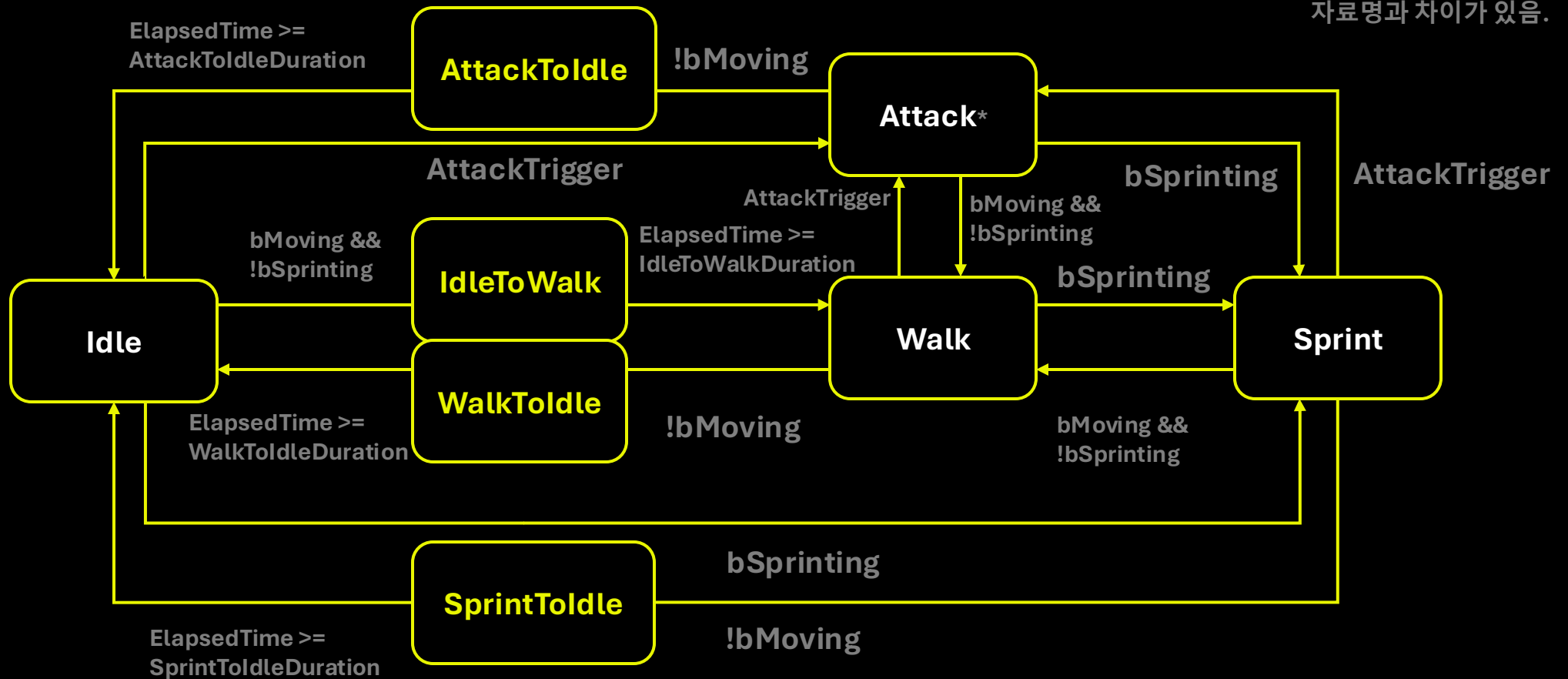
```
LocomotionStateMachine->AddBoolTransition(  
    FName("Idle"),  
    FName("Sprint"),  
    FName("bSprinting"),  
    EAnimCompareOp::IsTrue,  
    true,  
    LocomotionBlendTime,  
    20,  
    EAnimBlendEaseOption::EaseInOut);  
.  
.  
.
```

4. Attack0 | Locomotion을 끊을 때

```
for (const FName& StateName : LocomotionStates)  
{  
    LocomotionStateMachine->AddTriggerTransition(  
        StateName,  
        NAME_HeavyAttack,  
        NAME_HeavyAttack,  
        LocomotionBlendTime,  
        200,  
        EAnimBlendEaseOption::EaseInOut);  
.  
.  
.
```

Animation State Machine

Runtime AnimGraph for **Gwen**



* 실제로는 Attack이 LightAttack
과 HeavyAttack으로 구분되어 있
음.
* 이해를 돕기 위해 실제 코드상의
자료명과 차이가 있음.

Animation State Machine

Lua에서 구현된 State Machine (일부)

```
function AnimTest:ConfigureLocomotionStateMachine()
```

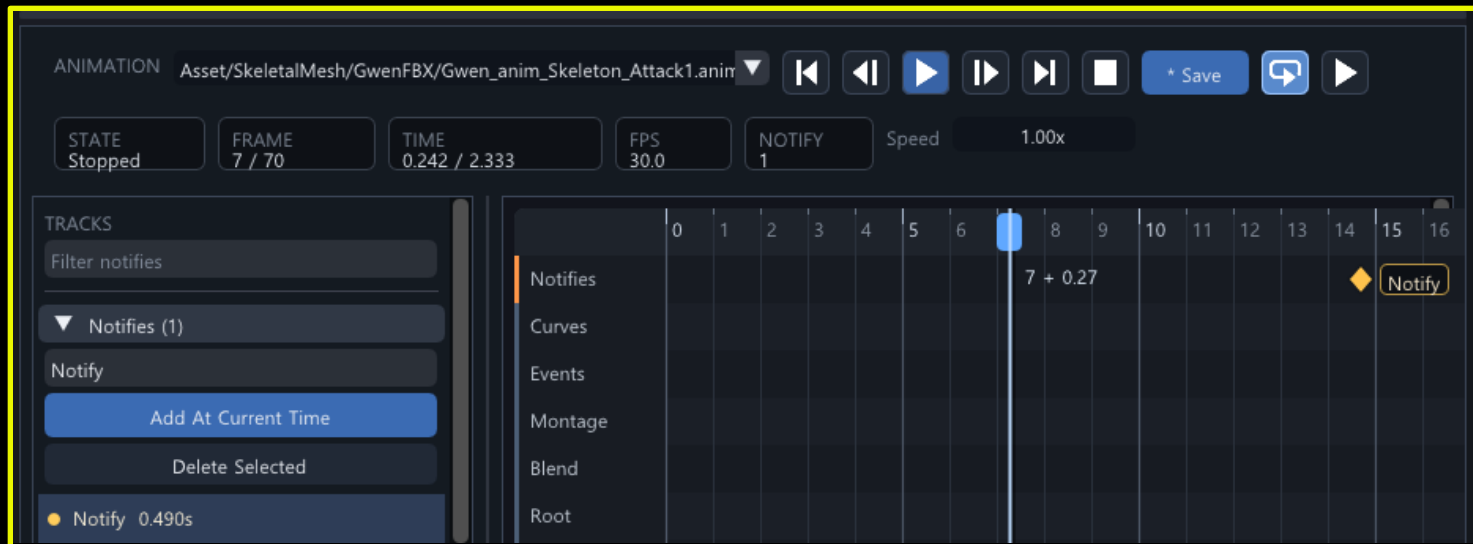
```
    local machine = CreateAnimStateMachineAsset()  
    machine:SetEntryState(N.Idle)  
    machine:AddStateWithPath(N.Idle, N.Idle, paths.Idle, true)  
    machine:AddStateWithPath(N.IntoRun, N.IntoRun, paths.IntoRun, false)
```

```
    machine:AddParameter("bMoving", "Bool")  
    machine:AddParameter("bSprinting", "Bool")  
    machine:AddParameter("StateElapsedTime", "Float")
```

```
    machine:AddBoolTransition(N.Idle, N.Homeguard, "bSprinting", "IsTrue", true, blend, 20, "EaseInOut")  
    machine:AddBoolTransition(N.Idle, N.IntoRun, "bMoving", "IsTrue", true, blend, 10, "EaseInOut")  
    machine:AddBoolTransition(N.IntoRun, N.Homeguard, "bSprinting", "IsTrue", true, blend, 110, "EaseInOut")  
    machine:AddBoolTransition(N.IntoRun, N.WalkToIdle, "bMoving", "IsFalse", false, blend, 100, "EaseInOut")  
    machine:AddFloatTransition(N.IntoRun, N.Run, "StateElapsedTime", "GreaterEqual", into_run, blend, 10, "EaseInOut")  
    machine:AddBoolTransition(N.Run, N.Homeguard, "bSprinting", "IsTrue", true, blend, 110, "EaseInOut")  
    machine:AddBoolTransition(N.Run, N.WalkToIdle, "bMoving", "IsFalse", false, blend, 100, "EaseInOut")
```

AnimNotify

사전 제작된 여러 이벤트를 애니메이션의 특정 시간에 **트리거**하기 위한 기능.



Unreal Reflection Property System

Reflection System이란?

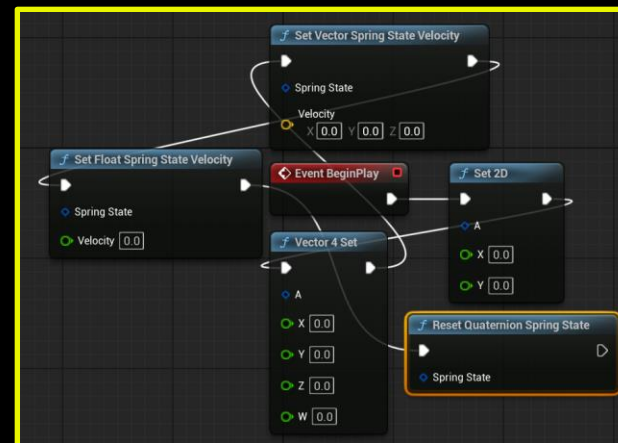
프로그램이 실행시간에 자기자신을 조사하는 기능.

왜 쓰는가?

런타임 단계에서 메타데이터를 이용하기 위해서이다.

어디에 쓰는가?

에디터, 직렬화, GC, 블루프린트 등 다양한 곳에서 쓰인다.



Blueprint

Inspector



C++ RTTI를 사용하지 않는 이유

1. C++ RTTI의 메타데이터 범위

- 런타임 타입 식별
- 상속 관계 기반 캐스팅

2. Unreal에서 필요로 하는 메타데이터 범위

- 멤버 정보
- 소유 클래스
- 함수 목록 등

즉, Unreal에서는 특별한 런타임 descriptor가 필요하다.

Reflection 동작 흐름



마크업

UCLASS, UPROPERTY, UFUNCTION 같은 매크로를 사용해

UHT가 어떤 C++ 타입과 멤버를 리플렉션 대상으로 처리해야 하는지 표시한다.

UCLASS / UPROPERTY / UFUNCTION

→ 클래스, 변수, 함수를 리플렉션 대상으로 표시한다.

.generated.h include

→ UHT가 생성한 코드를 현재 헤더에 포함한다.

GENERATED_BODY()

→ UHT가 생성한 선언 코드가 클래스 내부에 들어갈 위치를 제공한다.

Generator

C++ 헤더를 빌드 전에 스캔해서 UClass, UPROPERTY 마커를 찾고,
UClass / FProperty 메타데이터 코드로 변환한다.

```
#include "AReflectionTestActor.generated.h"

UCLASS()
class AReflectionTestActor : public AActor
{
    GENERATED_BODY(AReflectionTestActor, AActor)

public:
    UPROPERTY(Edit, LuaRead, LuaWrite)
    float Speed = 0.0f;

    UPROPERTY(Edit, LuaRead, LuaWrite)
    float Health = 0.0f;
};
```

```
Class.AddProperty(MakeProperty<FFloatProperty>(
    "Speed",
    offsetof(AReflectionTestActor, Speed),
    sizeof(float),
    EPropertyFlags::Read |
    EPropertyFlags::Write |
    EPropertyFlags::Edit |
    EPropertyFlags::LuaRead |
    EPropertyFlags::LuaWrite));
.
.
```

Showcase

김태현

김호준

김기훈